# Gradient Bench v3 — architecture and technical reference

This document explains what every file in `v3/` does, how the pieces fit together, and why each technical decision was made. It was written against the code on the `v3-shape-aware` branch on 2026-09-16; where a number appears here it was read from the source, not from memory. File references are relative to `v3/`.

Read it top to bottom once. After that, section 3 (the map) and section 12 (the v3 changes) are the ones to come back to.

What comes next (v4, a changeable objective chosen from the analyst's own ranking) is in `ROADMAP.md`.

---

## 1. What the program is

Gradient Bench is a closed-loop method-development tool for HPLC gradient elution. One analyst at one bench runs a **campaign**: the app proposes a gradient method, the analyst runs it on the instrument, uploads the exported chromatogram, checks the peaks the picker found, and records the result. A Bayesian optimiser refits a Gaussian process on every recorded run and proposes the next method. A fixed instrument-check method is re-run on an enforced cadence so instrument drift can be told apart from chemistry.

It is a single Flask process serving a pre-built React interface. There is no database, no account, no network dependency: a campaign is a folder on disk, and the frontend bundles its own fonts because the PC beside an HPLC often has no internet.

### The four-step loop, which is the whole product

```
suggest ──► upload ──► pick ──► record ──► (refit) ──► suggest …
   │           │          │          │
   │           │          │          └ writes one row per injection to the workbook,
   │           │          │            refits the model before/after, clears the loop
   │           │          └ runs the peak picker, analyst corrects it if wrong
   │           └ copies the ASCII trace into the campaign folder
   └ cold-start seed, model proposal, or the instrument check — decided by next_step()
```

Every fifth design method the loop interrupts to demand the instrument check instead, and the very first thing any campaign runs is that check (the "anchor").

---

## 2. Technology and why

| Layer | Choice | Why |
|---|---|---|
| Backend | Python 3.9+, Flask | One process, one machine, JSON only. Flask does routing and nothing else. |
| Modelling | PyTorch, GPyTorch, BoTorch `>=0.18, <0.19` | BoTorch supplies the GP, the priors, the acquisition functions and the constrained acquisition optimiser. The version is pinned because BoTorch's *default* kernel and priors have changed between releases, and a campaign fitted under one and refitted under another is not the same campaign. |
| Peak picking | numpy, scipy, pybaselines | Vendored picker `hplc_picker` 3.0.0, self-contained. |

| Layer | Choice | Why |
|---|---|---|
| Record | openpyxl / pandas workbook | An `.xlsx` a chemist can open without the app. |
| Frontend | React 18 + Vite 5, no UI library | Built into `gradient_bench/api/static/`; the Python package ships the built bundle so no Node is needed to run. |
| Tests | pytest, 147 tests | Golden files against 20 real chromatograms plus unit tests for every layer. |

Long calls (a GP fit, a picker pass, an export) run as in-process background threads (`jobs.py`) that the page polls every 250 ms, so a fit never leaves a button dead.

---

## 3. The map

```
v3/
  README.md, CHANGELOG.md, VERSION (1.1.0), ARCHITECTURE.md (this file)
  hplc_picker.py                3-line shim: `import hplc_picker` -> core/hplc_picker
  testdata/                     21 real chromatograms + their sheet (golden files, replay)
  docs/screenshots/
  frontend/                     React source; `npm run build` writes
../gradient_bench/api/static
  gradient_bench/
    __init__.py                 APP_VERSION = "1.1.0", VERSION_TAG = "version 3"
    loop.py                     THE ENGINE: next_step / suggest / add_upload / pick / record
    jobs.py                     background threads the page polls
    rescore.py                  re-measure every stored trace under new picker settings
    export.py                   workbook + 5-page PDF report into the campaign folder
    setup_doc.py                the printable "Method" page, assembled from running code
    testdata.py                 locates the bundled corpus
    core/                       THE SCIENCE — imports nothing from store/ or api/
      space.py                  the design spaces (ramp4, steps8), constraints, the gate
      shape.py                  the shape-aware (Wasserstein) kernels          [v3]
      optimiser.py              surrogate, noise floor, acquisition, proposal
      uncertainty.py            fixed evaluation grid, before/after audit, slice panels
      crf.py                    the objective, frozen at 1.0.0
      hplc_picker.py            the peak picker (1976 lines) — the ONLY copy
      picker.py                 adapter: campaign vs per-run config, the record row
      drift.py                  reference schedule, PASS/WATCH/HALT, drift adjustment
      health.py                 six diagnostic channels read from reference runs
      budget.py                 method budget, extend / finish / reopen, bench-time cost
    store/                      THE RECORD — a campaign is a folder
      campaign.py               Campaign class: create/open/duplicate, derived state
      sheet.py                  the workbook (Data | Campaign | Config), atomic writes
      state.py                  state.json — the in-flight PendingRun
      history.py                uncertainty_history.csv
      migrate.py                bring an older folder forward (additive only)
    api/
      app.py                    Flask routes: thin JSON translation over loop/store
      static/                   the built frontend
    scripts/
      verify_install.py         11 install checks
      replay_corpus.py          headless end-to-end over the corpus (--kernel)
    tests/test_core.py          147 tests
    golden_files.json           adjudicated expectations for the corpus
```

The dependency direction is strict: `core/` knows nothing about folders or HTTP; `store/` knows about folders and imports `core/`; `loop.py` orchestrates both; `api/app.py` only translates JSON to calls. Nothing in `core/` can be broken by a change to the API.

## 4. A campaign is a folder

```
<wherever the analyst chose>/<slug>/
    master_methods.xlsx      Data | Campaign | Config — the source of truth
    traces/                  every uploaded ASCII file, kept, named m03r1.TXT / ref02.TXT
    backups/                 timestamped workbook copies (20 kept)
    reference_grid.npy       the fixed evaluation grid, cached
    uncertainty_history.csv  one row per recorded method — what the MODEL did
    state.json               the in-progress loop, for resume
    .gradient_bench          marker: schema version
```

**Why a folder and not a database.** Portability and legibility. Copy the folder to a USB stick and it opens on another machine with its full history; open the workbook in Excel and every number is there. Identity is the folder path, which kills a class of bug the historic notebooks had (a tag string that two campaigns could share).

**The workbook** (`store/sheet.py`) has three sheets:

- `Data`: one row per **injection**. Columns are `ID_COLS` (method, replicate, source, run_order, recorded_at) + `PARAM_COLS` (the eight absolute gradient columns `t1 t2 t3 phi1 phi2 phi3 phi4 T`) + every picker measurement and trace-health descriptor
- `PROVENANCE_COLS` (CRF, crf_version, picker_version, picker_config, picker_deviates, manual_humps, trace_file, model_dim, notes). Provenance is stamped **per run**, not per campaign, because picker settings legitimately change mid-campaign.
- `Campaign`: metadata (analyst, sample, column, mobile phase, flow, detector, instrument) plus lifecycle (budget, finished flag, the history of edits and extensions).
- `Config`: the scientific settings — space, kernel, acquisition, priors note, picker defaults, library versions at creation. Meant not to change during a campaign.

Writes are atomic (temp file + rename) with a backup taken first.

**Naming.** Design runs are `m03r1`, `m03r2` (method 3, replicate 1 and 2); reference runs are `ref01`, `ref02`. `method_base()` strips the replicate suffix, and that is how replicate groups are found for the noise estimate.

**Two spaces, one sheet.** A single ramp writes `t1 = duration, t2 = t3 = 0, phi1 = start, phi2 = end`. A three-segment method fills all eight columns with node times and node compositions. `space_for_row()` reads `t2 == t3 == 0` as "ramp", which is how a mixed or an old folder reads back correctly without a schema change.

## 5. The loop engine (`loop.py`)

Every gate lives here, in one function, so the API cannot drift between endpoints.

### `next_step(campaign)` decides what the analyst sees

Checked in this order:

1. **A pending run always finishes.** If `state.json` holds a method, the next step is whichever of upload / pick / record is incomplete, whatever else is true. A closed or out-of-budget campaign can still record the traces it already paid for.
2. **Finished** — the analyst closed the campaign. Blocked.
3. **Budget exhausted** — blocked; the message also says if the instrument is failing its check, so nobody buys runs for a broken machine.

4. **Reference due** — the anchor (first ever) or the cadence check. Enforced, not advised.
5. **HALT** — the drift verdict blocks proposing.
6. **Cold start** — while fewer than `n_seed` (default 5) methods exist, or fewer than two scored runs, serve a pre-generated spread-out method with no model.
7. **Proposal** — fit the GP and optimise the acquisition.

**`suggest` → `add_upload` → `pick` → `record`**

- `suggest` persists a `PendingRun` (`store/state.py`) to `state.json`: kind, the method vector `u`, phase (`anchor | reference | cold_start | optimise`), replicate count, and, for a model proposal, the posterior mean and SD at that point. Feasibility is checked before persisting.
- `add_upload` copies the trace into `traces/` under its final name and appends the path.
- `pick` runs the picker on every uploaded trace with the campaign config plus any per-run overrides or hand-drawn unresolved regions, and returns the review payload (trace, peaks, categories, humps, CRF). Re-run on every change; nothing is cached.
- `record` re-checks feasibility **first** (a corrupt `state.json` must never reach the workbook), then requires every replicate uploaded and picked, then writes one `Data` row per replicate, clears `state.json`, and runs the uncertainty audit (section 9).

**Why replicates are mandatory by default.** Two injections at identical settings differ only by noise. That is the only direct measurement of σ available, and without it the GP cannot tell "smooth surface, large noise" from "wiggly surface, small noise".

---

## 6. The objective (`core/crf.py`)

```
CRF = n_clean_peaks × (1 − hump_time_fraction)²          version 1.0.0, maximised
```

`n_clean_peaks` is the count of cleanly resolved peaks; `hump_time_fraction` is the share of the run under an unresolved region. The square makes the penalty negligible for a small hump and severe for a large one.

It was chosen by a 56-candidate study against 20 expert-ranked chromatograms (first on the composite; Kendall τ 0.857 against reviewers with an inter-rater ceiling of 0.937). It is **frozen**: a moving objective makes campaigns incomparable. Every recorded row carries `crf_version`.

**The structural guard.** The picker also records ~15 trace-health descriptors (peak width, tailing, noise, sampling rate…). They describe the *instrument*, and if they reached the objective a drifting detector could move the score the optimiser climbs. So `assert_objective_is_clean()` fails loudly if any of them ever appears in `CRF_COLUMNS`, and it runs on every analysis.

---

## 7. The design spaces (`core/space.py`) — new in v3

A `Space` object carries names, a box, linear constraints in absolute units, and the chemistry: how to draw the profile, how to map to and from the eight sheet columns, how to sample seeds, how to print a gradient table. The space is chosen at campaign creation and locked, like the kernel: a campaign's runs are points in one space.

### `ramp4` (default) — `[start_phi, end_phi, duration_min, T]`

| Parameter | Range |
|---|---|
| start_phi | 0.02 – 0.40 (fraction B) |
| end_phi | 0.10 – 1.00 |
| duration_min | 10 – 60 min |
| T | 25 – 60 °C |

One constraint, `end_phi ≥ start_phi + 0`. The minimum span is zero on purpose: an isocratic method is a legitimate answer.

### `steps8` — `[phi0, dphi1, dphi2, dphi3, dt1, dt2, dt3, T]`

A four-node pump timetable written in **deltas**. Nodes are the cumulative sums:

```
(0, phi0), (dt1, phi0+dphi1), (dt1+dt2, phi0+dphi1+dphi2), (dt1+dt2+dt3, phi0+Σdphi)
```

The first node is always at t = 0, so it needs no parameter. Writing in deltas puts monotonicity into the box (every delta ≥ 0) and nothing else is bounded per segment: a composition step of zero is a hold, a time step of zero is an instantaneous jump, both zero is an absent segment. So one setup reaches a ramp, a step gradient, a hold-then-ramp, and two- or three-slope gradients.

Constraints, all on sums:

| Constraint | Value |
|---|---|
| phi0 + Σdphi ≤ | 1.00 |
| Σdt ≤ | 30 min |
| Σdt ≥ | 1 min |

The 1-minute floor is technical, not a design choice: the profile is drawn on time divided by the duration, so a zero-length method is undefined. `phi0` keeps the shared 0.02–0.40 envelope; `T` is 25–60 °C.

**Why constraints are passed natively.** BoTorch's `optimize_acqf` accepts linear inequalities as `(indices, coefficients, rhs)` triples over the unit cube. `Constraint.normalised()` converts each absolute constraint into that form, so proposals are feasible **by construction**, not by rejection. The same objects drive the gate (`check()`), the vectorised mask for grids, and the Method page's printout.

**The gate.** `require_feasible()` is called at cold-start sampling, at acquisition output, and again at recording. Nothing reaches the instrument or the workbook without it.

**Cold-start sampling.** `ramp4` rejection-samples the box (87% feasible). `steps8` uses the paper's *window-relative* sampler: draw the start and end compositions, draw the interior compositions uniformly inside that window and sort them, draw the duration and the interior times likewise. Uniform box sampling would land ~98.5% of draws outside the sum constraints and would bias toward diagonal gradients.

**The instrument check is always a ramp**, whatever space the campaign searches: it is a fixed method the instrument repeats, not a point the model sees.

Module-level names (`P4_NAMES`, `check4`, `to_norm`, `initial_sample`…) are the ramp4 space's methods exposed as functions, because that is how every caller written before v3 spelled them. `for_vector(u)` picks the space from a vector's length (4 or 8), and `gradient_table(u)` / `require_feasible(u)` dispatch through it.

## 8. The surrogate and the acquisition (`core/optimiser.py`, `core/shape.py`)

### The Gaussian process

`fit_gp()` builds a BoTorch `SingleTaskGP` on inputs normalised to the unit cube with `Standardize(m=1)` on the targets, and fits by exact marginal log likelihood (`fit_gpytorch_mll`). One row per **run**; replicates repeat the same x.

**The priors are BoTorch's, always.** A hand-built covariance module silently discards BoTorch's hyperparameter priors and leaves lengthscales at unregularised maximum likelihood, which on a few dozen points sends some to infinity and collapses others. So every module is built through BoTorch's own `get_covar_module_with_dim_scaled_prior` (LogNormal `(√2 + ln d / 2, √3)` on lengthscales, `GreaterThan(0.025)`, no outputscale), and the Matérn-3/2 and shape-aware variants copy those priors off a module BoTorch built rather than retyping numbers.

### Kernel options (chosen at creation, locked)

| Key | What | Notes |
|---|---|---|
| `matern52` | Matérn-5/2 ARD over the inputs | **default**. Twice differentiable: a chromatographic response can turn over a short interval, and an RBF must either smooth that away or collapse a lengthscale. |
| `matern32` | Matérn-3/2 ARD | one notch rougher |
| `rbf` | squared exponential ARD | BoTorch 0.18's own default |
| `wasserstein` | exp(−d_W/ℓ) × RBF(duration, T) | the paper's construction, Eq. 17–18 |
| `wasserstein_matern` | exp(−d_W/ℓ) × Matérn-5/2(duration, T) | this app's variant |
| `wasserstein_plus` | exp(−d_W/ℓ) × ARD RBF(start, end, duration, T) | the paper's Wasserstein+, Eq. 19 |
| `wasserstein_plus_matern` | exp(−d_W/ℓ) × ARD Matérn-5/2(start, end, duration, T) | this app's variant |
| `default` | no covar module passed | whatever the installed BoTorch does |

### The shape-aware kernels (`core/shape.py`) — new in v3

After Boelrijk, Detlefsen, Dürholt and Haas, *Shape-aware Gaussian process kernels for closed-loop Bayesian optimization of gradients in automated LC method development* (ChemRxiv, 2026).

**The distance.** Each method is drawn as a profile on a normalised time axis (time divided by the method's own duration) with composition as a fraction of the maximum %B. The two axes are treated differently on purpose: time is per-curve, composition keeps its absolute level. The distance between two methods is the **area between the two profiles**, which for monotonic one-dimensional profiles is the Wasserstein-1 distance. It is at most 1 and is zero only for identical profiles.

**Exact evaluation.** Both profiles are interpolated onto the union of their breakpoints. On each interval the difference is linear, so if it differs by `p` at the start and `q` at the end the area is `|p+q|/2` when they do not cross and `(p²+q²)/(2|q−p|)` when they do. Because a `steps8` profile may contain a vertical step, the difference can jump *at* a breakpoint; the routine therefore reads each interval at its quarter points (always

interior) and extrapolates exactly to the ends. No quadrature, no epsilon. Implemented twice: in numpy (`profile_w1`, for tests and explanations) and in torch (`_torch_w1`, batched over every pair and differentiable, which the acquisition optimiser needs).

**The kernel.** `exp(−d_W/ℓ_W)` is the Laplacian kernel in an L1 metric, positive definite for any $\ell > 0$. `WassersteinProfileKernel` (a GPyTorch `Kernel` with one lengthscale) maps normalised inputs to profiles itself, so it works in either space. ℓ_W carries BoTorch's dimension-scaled prior with D = 1 (the paper's choice) and the same 0.025 floor.

**The second factor.** The time normalisation discards the duration, and the profile never contained the temperature, so the shape term is multiplied by a kernel over those (`REST_FEATURES`), or, in the plus forms, over start %B, end %B, duration and T (`PLUS_FEATURES`). In `ramp4` those are the inputs; in `steps8` they are linear functions of the inputs (`end = phi0 + Σdphi`, `duration = Σdt`), so `LinearFeatureKernel` applies an affine map `f = xA + b` (from `Space.features()`) and hands the features to a BoTorch-built RBF or Matérn-5/2. The same four feature names therefore appear in either space.

**Why this matters most in `steps8`.** A four-node timetable can write one profile many ways (slide a node along the line joining its neighbours). A coordinate kernel has to learn from runs that those timetables are the same experiment; the Wasserstein distance between them is exactly zero. In `ramp4` there is no such redundancy, and the shape kernel only changes what "nearby" means.

**What is reported.** Coordinate kernels report one lengthscale per input. Shape-aware kernels report `shape` (in units of area, at most 1) plus one per feature. The names come from the fitted module (`kernel_lengthscales`) and, without torch, from the key (`lengthscale_names_for`), so the history file and the Model tab agree.

### The noise floor

Replicate scores of each method are pooled into a within-method σ (`pooled_within_sd`, in CRF units, with degrees of freedom). Until a replicate pair exists a fallback of 1.0 CRF is used and labelled. An exact tie (σ = 0) is treated as *unmeasured*, not zero, because the objective is driven by an integer peak count.

`Standardize` divides targets by their SD, so the floor must be converted: `floor_z = σ² / var(y)`. It is installed as the **lower bound** of the likelihood's noise (the prior stays BoTorch's). `floor_z ≥ 1` means replicate noise is as large as the whole campaign's spread; the Model tab says so and tells the analyst not to lower it.

### The acquisition

| Key | Class | When |
|---|---|---|
| `qlognei` | qLogNoisyExpectedImprovement | **default**: the incumbent is a noisy measurement; the log form keeps a gradient under a flat posterior |
| `qlogei` | qLogExpectedImprovement | a very quiet instrument |
| `ucb` | qUpperConfidenceBound, `beta` | scouting; beta is the analyst's to defend |
| `logpi` | LogProbabilityOfImprovement | late-stage refinement, greedy |

`suggest_next()` optimises with `optimize_acqf` (default 10 restarts, 512 raw samples) over the unit cube with the space's constraint triples, un-normalises, and gates the result.

**Run order as a covariate** is implemented (a fifth or ninth input pinned at the next run's position via `FixedFeatureAcquisitionFunction`) but off by default: it gives drift somewhere to live in the posterior, but leaves it in qLogNEI's yardstick, and it earns no interface until a leave-one-out comparison on a recorded campaign says it should.

## 9. Learning diagnostics (`core/uncertainty.py`, `store/history.py`)

After every recorded method the model is fitted on the data before and after it, and both posteriors are evaluated over a **fixed, cached evaluation grid** (512 scrambled-Sobol points inside the gate, one per space, `reference_grid.npy`). The grid is fixed on purpose: if the set of points moved between runs, the learning curve would be an artefact of resampling.

`UncertaintyStep` records mean and max posterior SD before/after, the mean absolute change in posterior mean, the SD at the incumbent and at the new point, the fitted noise, the floor, and every lengthscale. `history.py` appends it to `uncertainty_history.csv`, widening the header if a new `ls_<name>` column appears (a shape-aware campaign has `ls_shape`; a `steps8` campaign has `ls_phi0`… or the feature names). The Model tab **reads this file and never refits**, so it works with torch absent and opening a tab never costs a GP fit.

`campaign_reading()` combines three signals: rising fitted noise + flat posterior SD + falling reference = drift being absorbed as noise, and the reading says to stop and fix the instrument rather than propose again.

`slice_posteriors()` sweeps one parameter across its range with the others pinned at the incumbent — a question a chemist can answer — and marks sweep points the constraints reject.

## 10. The peak picker (`core/hplc_picker.py`, `core/picker.py`)

`hplc_picker` 3.0.0 is vendored and self-contained; the root `hplc_picker.py` is a shim so `import hplc_picker` still works. It defines what a peak *is*:

1. Load the two-column ASCII trace.
2. Baseline correction with arPLS (`pybaselines`); the stiffness is quoted at a reference sampling rate and rescaled to the trace's own rate.
3. Noise estimate by differencing at a fixed **time** lag; every window is in minutes and converted to samples against the trace's median dt, so two traces exported at different rates are treated the same.
4. Smoothing (Savitzky–Golay).
5. Prominence detection: the gate is `snr` × noise. This pass defines **clean** peaks.
6. Second-derivative pass with fit gating for **shoulders** (Exponentially Modified Gaussian fits, F-test gated).
7. Unresolved-region (**hump**, UCM) detection: a stretch where the lower envelope stays raised (`hump_min_floor_ratio`, `hump_min_span_abs`).
8. Categorisation: a pick inside a hump is `hump`; otherwise `clean` if the prominence pass found it, else `shoulder`. **Only clean peaks count toward the CRF.**
9. EMG deconvolution and resolution scorecards (diagnostic columns).
10. Trace-health descriptors (diagnostic, structurally barred from the objective).

`core/picker.py` is the adapter. Two levels of configuration: the **campaign** config (in `Config`) and a **per-run override** (the analyst correcting a failed estimate on one chromatogram, recorded on the row as `picker_deviates`). Hand-drawn unresolved regions are a separate flag, `manual_humps`. Reference runs refuse per-run overrides: they are a differential measurement, and retuning one turns a picking artefact into fake drift.

Three thresholds are exposed as primary controls (S/N gate, hump floor ratio, minimum hump span), each stating what raising and lowering it does; the rest are advanced. Changing the campaign config re-scores every stored trace (`rescore.py`), with a preview plan first, so the CRF column keeps meaning one thing.

## 11. Drift, the instrument, and the budget

### The control chart (`core/drift.py`)

One method held fixed, run first (the **anchor**, `ref01`) and again every `REFERENCE_EVERY = 5` design methods. Its movement can only be the instrument.

| Verdict | Rule |
|---|---|
| WATCH | 1.5 CRF below the anchor, or no reference at all (unmeasured is not clean) |
| HALT | 3.0 CRF below the anchor, or 3 consecutive falls |

A HALT blocks proposing; the instructions are to service and re-anchor, treating everything before the service as a separate block. Reference runs are excluded from the model structurally (`training_set()` filters on `source`).

**Drift-adjusted scores** (`drift_adjust`: raw minus the interpolated reference offset) exist but are off by default and **never** fed to the model: adjusting assumes the drift is reversible, and nothing can check that. The toggle changes what is reported as best, which can produce a "phantom incumbent" (raw-best and adjusted-best differ); the app says so.

### Six diagnostic channels (`core/health.py`)

Read from reference runs only: peak width (implicates the column), tailing (active sites or a void), retention (mobile phase or temperature), signal and area (injection or sample age), baseline and noise (detector or mobile phase), acquisition (the data system). Each reading is offered as a hypothesis. The acquisition channel detects that the *acquisition itself* changed — a different run length or sampling rate — which invalidates every other panel.

### The budget (`core/budget.py`)

Denominated in **unique design methods** (default 40); replicates and instrument checks cost none, and the bench-time conversion (injections, hours at 30 min a run) is always shown. Exhaustion blocks proposing only. Extending requires a reason and is written to the `Campaign` sheet's history; so are finish and reopen.

## 12. What changed in v3, and why

| Change | Where | Why |
|---|---|---|
| Design space is a per-campaign, locked choice: `ramp4` or `steps8` | `core/space.py`, `store/campaign.py`, wizard | A single ramp cannot express a hold or a bend. The three-segment timetable in deltas can, with only sum constraints. |
| `Space` objects with absolute constraints and a `normalised()` converter | `core/space.py` | One definition drives the gate, the sampler, the grid mask, the acquisition optimiser and the Method page. |
| Four shape-aware kernels | `core/shape.py`, `core/optimiser.py` | The paper's Wasserstein and Wasserstein+ forms, each with the paper's RBF or a Matérn-5/2 second factor. |

| Change | Where | Why |
|---|---|---|
| General, exact piecewise-linear profile distance in numpy and torch | `core/shape.py` | Needed for four-node profiles with vertical steps; the ramp closed form is kept for tests. |
| `LinearFeatureKernel` | `core/shape.py` | The endpoint features are sums of inputs in `steps8`; the kernel derives them so the same names appear in both spaces. |
| Lengthscale names come from the fitted module | `optimiser.kernel_lengthscales`, `FitResult.lengthscale_names` | A shape-aware fit reports `shape` plus features, not one per input. |
| Every optimiser and uncertainty function takes a `space` | `core/optimiser.py`, `core/uncertainty.py` | Dimension, bounds and constraints are read from it; the ramp remains the default argument so old callers are unchanged. |
| Grid regenerated if its width does not match the space | `Campaign.grid()` | A ramp grid must never be reused by a steps8 campaign. |
| Per-row method summaries in the API | `/api/runs`, `/api/methods`, `/api/trace` | A ramp and a timetable both have a start, an end, a duration and a segment count; the tables read those, not raw sheet columns. |
| Wizard gains the design-space choice; Model/Setup/Method pages name the space | `frontend/src/tabs/*` | Three locked choices instead of two. |
| Instrument check stays a ramp | `store/campaign.create` | It is a fixed method the instrument repeats, not a point the model sees. |
| `--kernel` on the corpus replay | `scripts/replay_corpus.py` | Headless exercise of any surrogate. |

**What was checked.** On a smooth synthetic response with 0.5 CRF noise, leave-one-out $R^2$ at 30 runs was +0.81 / +0.68 / +0.83 for Matérn / Wasserstein / Wasserstein+, with the shape lengthscale settling near 0.16. On the bundled 15-method corpus every kernel has negative leave-one-out $R^2$ and the shape lengthscale collapses to its floor; a marginal-likelihood profile showed that is what the data prefers, not an optimiser artefact. The corpus is too small and too noisy to rank kernels. Nothing is yet validated on a real campaign; the Model tab's learning curve is the test.

---

## 13. The HTTP API (`api/app.py`)

Every response is `{"ok": true, ...}`; errors are `{"ok": false, "error", "kind"}` with 400 for `ValueError`/`RuntimeError`. NaN and infinity are nulled. One campaign is open per process. Long calls return `{"job": {...}}` and the client polls `GET /api/jobs/<id>`.

| Route | Does |
|---|---|
| `GET /api/version` | app version, library versions, whether torch imports |

| Route | Does |
|---|---|
| `GET /api/optimiser/options` | spaces, surrogates, acquisitions, defaults, reference presets, budget default |
| `GET /api/campaigns`, `GET /api/browse` | recent campaigns; server-side folder browser |
| `POST /api/campaigns/create` | folder + workbook from the wizard body (info, reference method, replicates, seeds, budget, cadence, space, kernel, acquisition, params) |
| `POST /api/campaigns/open\|duplicate\|archive\|reveal` | lifecycle |
| `GET /api/state` | everything the chrome and the Run tab need: budget, campaign summary, next step, pending run (with gradient table, summary, T), drift, schedule |
| `POST /api/suggest\|upload\|pick\|record\|discard` | the loop |
| `GET /api/budget`, `POST /api/budget/extend`, `POST /api/campaign/finish\|reopen` | budget |
| `GET /api/runs`, `GET /api/methods`, `GET /api/trace/<method>`, `GET /api/reference` | results |
| `POST /api/export` | workbook and/or PDF into the campaign folder |
| `GET\|POST /api/campaign/info` | metadata with a correction log; material changes need a reason |
| `GET /api/setup` | the Method document |
| `GET\|POST /api/migrate` | inspect / apply schema migration |
| `GET /api/model` | Model tab: trend, ARD table, noise readout — read from the history file, never a fit |
| `GET /api/drift`, `POST /api/toggle` | Instrument tab; the drift-adjusted toggle |
| `GET /api/picker/controls`, `POST /api/picker/rescore` | picker settings; re-score with preview |

## 14. The frontend (`frontend/src/`)

Plain React, no state library, no UI library. `App.jsx` holds one `state` object from `/api/state`, re-fetched after every mutation (`reload()`), and seven WAI-ARIA tabs with arrow-key navigation. Tabs other than Campaign are disabled until a campaign is open.

| File | Role |
|---|---|
| `api.js` | `get`, `post`, `upload`, and `runJob` (polls a job every 250 ms) |
| `hooks.js` | `useAsync` (busy label + error), `useLocal` (persisted UI prefs), theme, live-region announcements |
| `glossary.js` | ~60 terms; `Term.jsx` attaches a definition where a term appears |
| `tabs/Campaign.jsx` | recent campaigns, folder browser, the six-step wizard (folder, campaign, instrument check, plan, **model** = space + surrogate + acquisition, review) with a live summary rail |
| `tabs/Run.jsx` | the four-step loop: one state banner, the gradient table, upload drop zone, chromatogram review with peak categories and draggable unresolved regions, picker thresholds, record |

| File | Role |
|---|---|
| `tabs/Results.jsx` | best-so-far, the methods ledger (grouped replicates, segments column), every run, compare two runs |
| `tabs/Model.jsx` | the fitted-with tiles, the learning curve, the lengthscale table (with the shape-lengthscale note), the noise readout |
| `tabs/Instrument.jsx` | the control chart and the six channels |
| `tabs/Setup.jsx` | metadata editing, the locked model tiles, picker settings with re-score preview |
| `tabs/Method.jsx` | renders the setup document, printable |
| `components/Chromatogram.jsx` | the trace with peak marks and hump spans; drag to draw a region |
| `components/charts.jsx` | learning curve, control chart, lengthscale bars |
| `components/ui.jsx` | Banner, Tag, Tile, Field, Choice (radio card), GradientTable (any number of nodes), Meter, Skeleton |
| `styles/tokens.css`, `styles/app.css` | design tokens (light/dark/system), peak-category colours distinct from instrument-state colours, each with a shape |

`npm run build` writes into `gradient_bench/api/static/`; that folder is committed so the Python package runs without Node. The dev server proxies `/api` to port 5051.

---

## 15. Reports and documents

- **The Method page** (`setup_doc.py`, `/api/setup`) assembles a full methods statement from the running code and the campaign's workbook: identity, the objective, the design space (parameters, every constraint in absolute and normalised form, sampler, gate), replicates and the noise model with the campaign's own numbers, the surrogate (each kernel factor with its prior and constraint, introspected from a module BoTorch builds, without fitting), the acquisition, the drift monitor, the stopping rule, the picker, library versions, and the record. Its premise is that nothing on it is maintained by hand.
- **Export** (`export.py`) writes a standalone workbook and a five-page matplotlib PDF (summary, best separation with its method, every method, the reference chart, provenance) into the campaign folder.
- **Migration** (`store/migrate.py`) brings an older folder forward additively: it adds a missing column or default, never rewrites a measured value, infers `trace_file` only when a file of exactly the expected name exists, and backs up first. A folder without a `space` key reads as the ramp; one without a `kernel` key reads as `default`.

---

## 16. Testing and scripts

- `python -m pytest gradient_bench/tests -q` — 147 tests. Golden files adjudicate the 20 corpus traces; the space tests check constraint normalisation against the absolute definition on random points, the sampler, and round trips through the sheet; the shape tests check the distance against quadrature, the metric axioms, vertical steps, the Gram matrix's eigenvalues, the priors, and that every kernel fits and proposes a feasible method in both spaces; the store tests create, record, duplicate and migrate campaigns.
- `python -m gradient_bench.scripts.verify_install` — 11 checks that the installed picker and libraries agree with what the objective was frozen against.

- `python -m gradient_bench.scripts.replay_corpus --trace-dir testdata/traces [--kernel …]` — scores the corpus from raw chromatograms, fits, proposes, runs the uncertainty audit and the drift gate, headless.

---

## 17. Decisions worth knowing before changing anything

1. **The objective is frozen.** Change it only by bumping `CRF_VERSION`; every row is stamped with the version that scored it.
2. **Priors are BoTorch's.** Do not build a covariance module by hand; copy priors off a module BoTorch built, as `matern32` and `shape.py` do.
3. **Space and kernel are locked per campaign.** There is no toggle by design; duplicate the settings into a new campaign instead.
4. **The GP always sees raw scores.** Drift adjustment is reporting only.
5. **Reference runs never reach the model, and never take per-run picker overrides.**
6. **The evaluation grid is fixed and cached.** Regenerate only when the space changes.
7. **Only clean peaks count**, and trace-health descriptors are structurally barred from the objective by an assertion that runs on every analysis.
8. **Do not narrow the box to suit a sample.** The box is the instrument envelope; let the optimiser find the dead regions.
9. **A pending run always finishes**, whatever the budget or the drift verdict says.
10. **The workbook is the record; the history CSV is derived.** The CSV can be deleted and regenerated; the run rows cannot.